

NO LANGCHAIN, NO LLAMAINDEX, NO MAGIC

Build RAG with Plain Python

Nine small functions and a provider SDK. Once you have written it yourself, every framework finally makes sense.

Write It Yourself First

MOTIVATION

A framework hides the decisions that decide your quality. Writing the pipeline once means you know exactly which knob to turn when answers get worse.

WITH A FRAMEWORK

- Working demo in 20 lines
- Chunking chosen for you
- Prompt hidden two layers down
- Debugging means reading library code

PLAIN PYTHON

- Every decision is visible
- You own chunking and retrieval
- You can print the exact prompt
- Stack traces point at your code

WHAT YOU WILL ACTUALLY BUILD



A loader and a chunker



An embedder and a store



Vector and keyword search



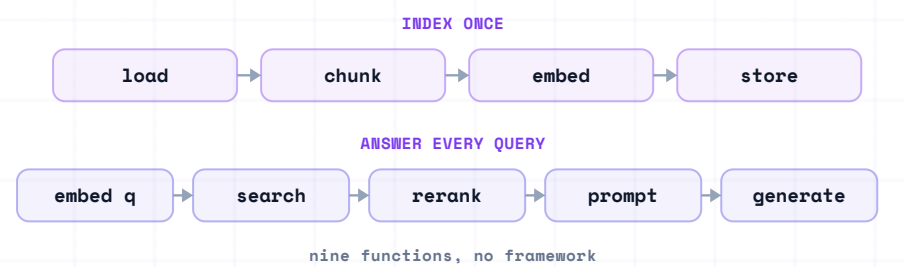
Prompt assembly and generation

You are not avoiding frameworks forever. You are earning the right to choose one on purpose.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

The Whole Pipeline

Two paths. One runs once when documents change, the other runs on every question. Keeping them separate is the first design decision.



PATH	RUNS	COST
Index	On ingest, or nightly	Slow and expensive, one time
Query	Per user question	Fast and cheap, every time

-  **Chunking**
-  **Hybrid search**
-  **Reranking**
-  **Prompt grounding**

Anything you can move from the query path into the index path is latency your users never pay for.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

What You Actually Need

DEPENDENCIES

Four packages. Nothing here is a framework, and you could drop two of them if you wanted to.

```
pip install openai numpy pypdf tiktoken
# optional but worth it
pip install sentence-transformers rank-bm25
```

PACKAGE	WHY
openai / anthropic	The model call, nothing more
numpy	Vector maths, this is your vector database at first
pypdf	Getting text out of PDFs
tiktoken	Counting tokens before you pay for them
sentence-transformers	Local embeddings and reranking
rank-bm25	Keyword scoring for hybrid search

You can build a genuinely good RAG system with numpy as the vector store. Add a database when you outgrow memory, not before.

Nothing on this list is a framework, and two of them are optional.

↓ [CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Load the Documents

STEP 1

Turn files into text plus metadata. The metadata you keep here is the metadata you can cite later, so keep more than feels necessary.

```
from pypdf import PdfReader
from pathlib import Path

def load(folder):
    docs = []
    for p in Path(folder).glob('*.pdf'):
        reader = PdfReader(p)
        for i, page in enumerate(reader.pages):
            text = (page.extract_text() or '').strip()
            if text:
                docs.append({'text': text, 'source': p.name,
                           'page': i + 1})
    return docs
```



Scanned PDFs need OCR,
not pypdf



Keep source and page or you
cannot cite

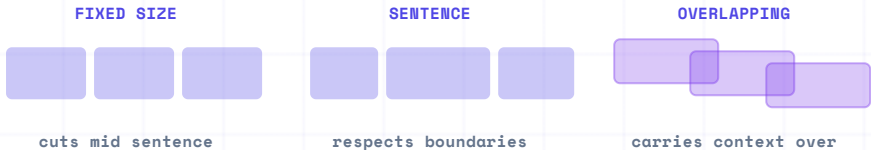
Garbage text here becomes garbage answers later, and it is almost impossible to diagnose from the other end of the pipeline.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Chunk the Text

STEP 2

The single highest leverage decision in the pipeline. Too big and retrieval is noisy, too small and the answer loses its context.



overlap is the cheapest quality win in the whole pipeline

```
def chunk(text, size=800, overlap=120):
    out, start = [], 0
    while start < len(text):
        end = start + size
        if end < len(text):          # back off to a boundary
            cut = text.rfind('. ', start, end)
            if cut > start: end = cut + 1
        out.append(text[start:end].strip())
        start = end - overlap
    return [c for c in out if c]
```

Start at 800 characters with 120 overlap. Then change one number at a time and measure, never both at once.

Code and tables need bigger chunks and structure aware splitting.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Embed the Chunks

STEP 3

Each chunk becomes a vector. Batch the calls, because one request per chunk will be slow and expensive for no reason.

```
import numpy as np
from openai import OpenAI
client = OpenAI()

def embed(texts, model='text-embedding-3-small'):
    vecs = []
    for i in range(0, len(texts), 100):          # batch of 100
        r = client.embeddings.create(model=model,
            input=texts[i:i+100])
        vecs += [d.embedding for d in r.data]
    V = np.array(vecs, dtype=np.float32)
    return V / np.linalg.norm(V, axis=1, keepdims=True)
```



Normalise once, then search is a dot product



Changing the model means re-embedding all of it

Use the same model for indexing and querying. Mixing them produces retrieval that looks fine and is quietly wrong.

Store It

STEP 4

Your first vector store is a numpy array and a list of dicts saved to disk. That is genuinely enough for a hundred thousand chunks.

```
np.save('vectors.npy', V)
json.dump(chunks, open('chunks.json', 'w'))

# later, at start up
V = np.load('vectors.npy')
chunks = json.load(open('chunks.json'))
```

SCALE	STORE
Under 100k chunks	numpy in memory, roughly 600MB at 1536 dims
Under 5M chunks	Postgres with pgvector and an HNSW index
Beyond that	A dedicated vector database

 Save vectors and chunks together, always

 Store a content hash to skip unchanged chunks

A numpy dot product over 100,000 vectors takes a few milliseconds. Most teams reach for a vector database far earlier than they need one.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Similarity From Scratch

MATHS

There is no mystery in vector search. It is one matrix multiply and an argsort.



$$\cos(\theta) = \frac{A \cdot B}{(|A| |B|)}$$

normalise both vectors first

and it is just a dot product

small angle = similar meaning

```
def search(query, V, chunks, k=12):
    q = embed([query])[0]          # already normalised
    scores = V @ q                  # cosine, one multiply
    idx = np.argsort(-scores)[:k]
    return [(chunks[i], float(scores[i])) for i in idx]
```



One matrix multiply, no index needed



Retrieve wide here, narrow it later

Retrieve 20 and rerank down to 4. Retrieving exactly what you need is how you end up missing the one chunk that mattered.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Keyword Search Too

BM25

Vectors miss exact terms: error codes, product names, part numbers. BM25 catches precisely what embeddings blur.

```
from rank_bm25 import BM25Okapi

tokenised = [c['text'].lower().split() for c in chunks]
bm25 = BM25Okapi(tokenised)

def keyword_search(query, k=12):
    scores = bm25.get_scores(query.lower().split())
    idx = np.argsort(-scores)[:k]
    return [(chunks[i], float(scores[i])) for i in idx]
```

Finds SKUs, codes and rare words

💬 Misses paraphrase entirely

🖋 Costs nothing, runs in process

+ Best used alongside vectors, never instead

If users search by identifier and get nothing back, this is the missing piece. No embedding model will memorise your part numbers.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Fuse the Two

HYBRID

Reciprocal rank fusion combines two ranked lists without needing their scores to be on the same scale. Ten lines, and it is the biggest single quality jump available.

```
def rrf(*ranked_lists, k=60):  
    scores = {}  
    for lst in ranked_lists:  
        for rank, (chunk, _) in enumerate(lst):  
            key = (chunk['source'], chunk['page'], chunk['text'])  
            scores[key] = scores.get(key, 0) + 1 / (k + rank + 1)  
    return sorted(scores.items(), key=lambda x: -x[1])
```



No score normalisation
needed



k = 60 is the standard
constant



Works with any number of
ranked lists



Deterministic and easy to
test

RRF needs no tuning and no score normalisation. It is the rare improvement that is both free and reliable.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Rerank the Survivors

PRECISION

Retrieve twenty, rerank, keep four. A cross encoder reads the question and each chunk together, which is far more accurate than comparing two separate vectors.

```
from sentence_transformers import CrossEncoder
ce = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

def rerank(query, candidates, top_n=4):
    pairs = [(query, c['text']) for c in candidates]
    scores = ce.predict(pairs)
    order = np.argsort(-scores)[:top_n]
    return [candidates[i] for i in order]
```



Fewer, sharper chunks
means less noise



And a smaller, cheaper
prompt



Costs roughly 50ms on CPU
for 20 pairs



The highest return
component after hybrid
search

A cross encoder is slow because it reads every pair properly. That is exactly why it is more accurate than comparing two vectors.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Build the Prompt

GROUNDING

This is where you force grounding and make citations possible. Tag every chunk with its source inside the context, because the model can only cite what it can see.

```
SYSTEM = 'Answer only from the context. If it is not there, say so.'
```

```
def build_prompt(query, chunks):  
    ctx = '\n\n'.join(  
        f'[{c["source"]} p{c["page"]}]\n{c["text"]}' for c in chunks)  
    return [{ 'role': 'system', 'content': SYSTEM },  
            { 'role': 'user',  
              'content': f'Context:\n{ctx}\n\nQuestion: {query}' }]
```



Source tags in context
enable citations



Tell it to answer only from
the context



Give it permission to decline



Cap the answer length in the
prompt

Give the model explicit permission to say it does not know. Without that sentence it will invent an answer rather than return nothing.

Generate the Answer

STEP 9

The last step, and the least interesting one. By the time you get here, quality is already decided by everything above.

```
def answer(query):  
    vec = search(query, V, chunks, k=20)  
    kw = keyword_search(query, k=20)  
    fused = rrf(vec, kw)[:20]  
    top = rerank(query, [c for c, _ in fused])  
    msgs = build_prompt(query, top)  
    r = client.chat.completions.create(  
        model='gpt-4o-mini', messages=msgs, temperature=0)  
    return r.choices[0].message.content, top
```



Temperature 0 for document
QA



Return the chunks too, for
citations



Set a timeout on the call



Retry once with backoff on
failure

This function is the whole system. Everything above it is a helper, and everything after it is measurement.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Stream It

LATENCY

The whole answer takes several seconds. Streaming does not make it faster, but it changes how slow it feels.

```
stream = client.chat.completions.create(  
    model='gpt-4o-mini', messages=msgs, stream=True)  
  
for chunk in stream:  
    delta = chunk.choices[0].delta.content  
    if delta:  
        print(delta, end='', flush=True)
```

STAGE	TYPICAL TIME
Embed the query	20 to 50 ms
Vector plus keyword search	5 to 30 ms
Rerank 20 candidates	50 to 200 ms
Generate the answer	1 to 4 seconds

Retrieval is a rounding error in your latency budget. Do not optimise it before you have optimised the answer length.

Streaming changes perceived latency, never real latency.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Measure Retrieval

EVALUATION

You cannot improve what you do not measure, and retrieval is measurable separately from generation. Do that first.

```
# 50 questions, each with the chunk ids that should come back
def recall_at_k(testset, k=5):
    hits = 0
    for q, expected in testset:
        got = {c['source'] for c, _ in search(q, V, chunks, k)}
    return hits / len(testset)
```

METRIC	ANSWERS
Recall at k	Did the right chunk come back at all
MRR	Was it near the top of the list
Faithfulness	Did the answer stay inside the context
Answer relevance	Did it actually address the question



50 questions is enough to start



Label the expected chunks by hand once

If recall at 5 is low, no prompt engineering will save you. Fix retrieval before you touch the generation step.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Caching and Budget

MONEY

Two caches remove most of your spend, and neither is more than a dictionary lookup.

```
import hashlib, json

def cache_key(query):
    return hashlib.sha256(query.lower().strip().encode()).hexdigest()

if (hit := cache.get(cache_key(q))):
    return hit # zero tokens spent
```

COST LINE	TYPICAL SHARE
Generation tokens	Roughly 90 percent of the bill
Embedding at query time	A fraction of a cent per query
Embedding at index time	One time, then only on updates
Reranking	Free if you run it locally

An exact match cache on normalised queries typically absorbs a quarter of production traffic for the cost of a dictionary.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Where This Breaks

LIMITS

This design is genuinely production ready up to a point. Here is exactly where each part gives out.

PART	BREAKS WHEN	REPLACE WITH
<code>numpy store</code>	Vectors exceed memory	pgvector or Qdrant
<code>Full scan search</code>	Beyond a few hundred thousand	An HNSW index
<code>In process BM25</code>	The corpus stops fitting in RAM	OpenSearch or Postgres FTS
<code>Rebuild on ingest</code>	Documents change often	Incremental upsert by hash



Watch memory, not query time, as your first signal



Plan the reindex path before you need it

Every one of these is a single component swap, because you wrote it as functions instead of adopting a framework. That is the payoff.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

What to Add Next

ROADMAP

In the order that actually returns the most quality per hour spent.

ADD	WHY IT IS NEXT
Hybrid search	Biggest jump, almost no cost
A reranker	Second biggest, still cheap
Query rewriting	Fixes vague and pronoun heavy questions
Metadata filters	Scopes by tenant, date and document type
A grading step	Detects bad retrieval before generating
Incremental indexing	Stops full rebuilds on every change

Do them in this order. Teams that start with query rewriting before hybrid search spend a week for a fraction of the gain.

AND WHAT TO SKIP



A bigger embedding model, usually



Switching vector database

Measure recall at 5 before and after each one, or you are guessing.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

When to Graduate

FRAMEWORKS

Frameworks earn their place when your control flow gets genuinely complicated. Not before, and not because of integrations.

STAY WITH PLAIN PYTHON

- One retrieval, one generation
- A single team owns it
- You want full visibility
- Latency and cost are tight

ADOPT A FRAMEWORK

- Loops and retry logic
- Human approval steps
- Many tools and branches
- Persisted, resumable state



LangGraph when the flow branches



LlamaIndex for exotic retrieval strategies



pgvector when memory runs out

Adopting a framework after writing this is a choice. Adopting one before is a guess about a problem you have not met yet.

The Whole System

RECAP

Nine functions, in the order you wrote them. That is the entire system.

FUNCTION	DOES
<code>load()</code>	Files to text plus metadata
<code>chunk()</code>	Text to overlapping pieces
<code>embed()</code>	Pieces to normalised vectors
<code>search()</code>	Query vector to top k by cosine
<code>keyword_search()</code>	BM25 over the same chunks
<code>rrf()</code>	Fuse two ranked lists
<code>rerank()</code>	Cross encoder picks the best few
<code>build_prompt()</code>	Context with source tags
<code>answer()</code>	Ties it together and calls the model

Roughly 120 lines. Small enough to read in one sitting, and small enough that you know exactly where every answer came from.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

RAG Without a Framework



Loading



Chunking



Embeddings



numpy as a store



Cosine similarity



BM25



Rank fusion



Reranking



Citations



Recall at k

Anyone can call a retriever.
Engineers have written
every line of one.

Save this and build it once from scratch.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT